

SAD Arquitectura

SAD - Documento de Arquitectura de Software

PawMily

****Versión:**** 1.0 · ****Fecha:**** 25 de agosto de 2026

1. Propósito

Describir la arquitectura del sistema PawMily (backend, web y app), patrones adoptados, stack y justificación.

2. Vista de contexto

[diagrama/codigo omitido en PDF - ver .md]

3. Patrón arquitectónico

3.1 Backend - Clean Architecture + DDD (ADR-001)

Se abandonó NestJS monolítico acoplado a favor de ****Express + TypeScript**** con capas:

Capa | Responsabilidad | Ubicación típica
Domain | Entidades, reglas, value objects | `src/domain`
Application | Casos de uso | `src/application`
Infrastructure | Prisma, storage, JWT | `src/infrastructure`
Interfaces | HTTP routes, middlewares | `src/interfaces/http`

****Contextos delimitados:**** Identity, Patients, Scheduling, Clinic, Media.

****Justificación:**** desacoplar clientes (web/Android), testabilidad, validación Zod en bordes, un solo contrato
`/api`.

3.2 Portal web - Cliente estático

SPA ligera por páginas HTML + módulos JS (`PawApi`). Sin framework pesado. Consume REST.

****Justificación:**** despliegue simple en Vercel, bajo costo, foco clínico.

3.3 App Android - Capas presentacional + repositorio

Activities/Fragments + `RemotePetRepository` + Retrofit/OkHttp. Sesión en `SessionManager` (EncryptedSharedPreferences).

****Justificación:**** rol owner-first, reutilización del contrato REST, notificaciones locales sin depender de FC
M en esta fase.

4. Stack tecnológico

Área | Tecnología

Runtime API | Node ? 20

Framework | Express 4

ORM | Prisma 5

Validación | Zod

Auth | jsonwebtoken + bcryptjs

BD | PostgreSQL (Supabase)

Deploy API | Railway

Web | HTML/CSS/JS

Deploy web | Vercel

Android | Kotlin, Retrofit, ZXing (barcode)

Seguridad HTTP | Helmet, CORS, rate-limit

5. Seguridad

- Passwords hasheadass (bcrypt).
- Access JWT de corta duración + refresh rotado/revocable.
- Autorización por rol y ownership (`vetId`, `ownerUserId`, `PatientAccess`, `LinkRequest`).
- Rate limiting en `/api` y rutas auth.
- Payload JSON limitado (p. ej. 2mb).

6. Despliegue

Ver diagramas en `Diagramas\_UML/Despliegue.svg` y guía en carpeta 03.

Ambientes:

- **Producción API:** Railway
- **Producción BD:** Supabase
- **Producción Web:** Vercel
- **Android:** APK/debug instalable apuntando a API prod

7. Decisiones clave

1. Un solo backend para vet y owner.
2. Código `PAW-?` + Code128 como identidad física.
3. Vinculación con solicitud/aprobación (no auto-link silencioso en flujo principal).
4. Fotos: data URL embebida o asset en storage.

8. Riesgos y deuda

Riesgo | Mitigación

Data URLs grandes en listados | Respuestas lean + downsample en Android

Backend sin git remoto | Inicializar remoto / backups 3-2-1

Sin rol admin SaaS | Manual de soporte operativo (carpeta 04)

Push cloud ausente | Notificaciones locales Android

9. Calidad

- Tests Jest (unit) en backend.
- Informes en `docs/` del backend (auditoría, seguridad, paridad Android).
- Health check público `/api/health`.